INTELLIGENT PROJECT MANAGEMENT SYSTEM FOR UNDERGRADUATE STUDENTS

INDIVIDUAL THESIS

S. A. I .K Kulathunga

(IT22109576)

Individual component: Predictive Risk Monitoring and Contribution Analytics

Group ID: R26-IT-095

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Information Technology

Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: S. A. Imansi Kavindi Kulathunga

Student ID: IT22109576

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Prof.Samantha Rajapaksha		
Co-supervisor: Ms.Lashika		

ABSTRACT

Undergraduate software projects require regular monitoring of team participation and delivery progress. However, manual progress reviews and isolated repository statistics may not reveal unequal contributions or emerging delays in time for intervention. This thesis presents the Predictive Risk Monitoring and Contribution Analytics component of the Intelligent Project Management System (IPMS). The component obtains development activity from GitHub and optionally obtains delivery indicators from Jira, computes individual Contribution Index (CI) scores, and classifies project risk as Low, Medium, or High. The implemented CI combines commit activity (30%), task completion (25%), collaboration (20%), issue resolution (15%), and code quality (10%), with weight renormalization when relevant activity is absent. A separate Analytic Hierarchy Process (AHP) exercise examines criterion priorities. For risk classification, 4,000 simulation-generated records with 12 activity and delivery features were divided into training and testing sets using an 80:20 split. Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting were compared. Logistic Regression achieved the highest reported test accuracy, 97.63%, with weighted F1 approximately 0.98. A FastAPI application provides project registration, Excel team upload, contribution analytics, risk dashboards, and email alerts. Repository-based demonstrations and a delivered High Risk notification illustrate the implemented workflow. The work establishes a functioning academic monitoring prototype; classifier performance is reported for the constructed evaluation dataset rather than independently observed future project outcomes.

Keywords: Intelligent Project Management System, Contribution Index, Project Risk Prediction, GitHub, Jira

ACKNOWLEDGEMENTS

I sincerely thank the academic supervisors and lecturers of the Faculty of Computing at the Sri Lanka Institute of Information Technology for their guidance throughout this research. I am grateful to my IPMS research group members for their collaboration in developing the integrated application. I also thank those who provided technical advice and feedback on the component, and my family for their continued encouragement and support.

CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 1
1.3 Problem statement 1
1.4 Research gap 2
1.5 Aim and objectives 2
1.6 Research questions or hypotheses 3
1.7 Scope limitations and assumptions 3
1.8 Individual contribution 3
1.9 Thesis structure 3
2 LITERATURE REVIEW AND RELATED WORK 4
2.1 Review method and scope 4
2.2 Domain and theoretical foundations 4
2.3 Related research 4
2.4 Existing systems products and techniques 5
2.5 Gap synthesis and conceptual framework 5
3 RESEARCH METHODOLOGY 6
3.1 Research design 6
3.2 Data participants or study materials 6
3.3 Experimental or study procedure 7
3.4 Measures and evaluation metrics 7
3.5 Data analysis 8
3.6 Ethics privacy security and responsible AI 9
3.7 Reproducibility and research artefacts 9
4 PRODUCT AND SOLUTION DESIGN 10

4.1 Requirements 10
4.2 Component architecture within the complete solution 10
4.3 Design alternatives and decisions 11
4.4 Detailed design 11
4.5 Standards and professional practice 12
5 IMPLEMENTATION INTEGRATION AND TESTING 13
5.1 Development environment and technology stack 13
5.2 Component implementation 13
5.3 Integration with the other components 14
5.4 Verification and software testing 14
5.5 Deployment and operation 14
6 RESULTS AND VALIDATION 16
6.1 Research results 16
6.2 Product and technical results 16
6.3 User or stakeholder validation 20
6.4 Integration results 20
6.5 Summary of findings 20
7 DISCUSSION 22
7.1 Interpretation of findings 22
7.2 Comparison with literature baselines and existing solutions 22
7.3 Research contribution 22
7.4 Product contribution 22
7.5 Limitations and threats to validity 23
7.6 Implications 23
8 COMMERCIALISATION AND SUSTAINABILITY 24
8.1 Value proposition and target users 24
8.2 Market and competitive position 24

8.3 Business or sustainability model 24
8.4 Scalability intellectual property and compliance 24
8.5 Contribution to the Sustainable Development Goals 25
9 CONCLUSION AND FUTURE DIRECTIONS 26
9.1 Conclusion 26
9.2 Achievement of objectives 26
9.3 Individual research and product contributions 26
9.4 Future directions 26
REFERENCES 28
GLOSSARY 29
APPENDICES 31
Appendix A Research instruments or extended results 31
Appendix B Academic integrity and AI use disclosure 31

LIST OF FIGURES

Figure 6.1 Risk prediction interface with Medium Risk classification

Figure 6.2 Contribution Index results for a four-member project

Figure 6.3 Analytics dashboard showing High Risk and project activity warnings

Figure 6.4 Project listing and Excel bulk upload interface

Figure 6.5 Delivered automated risk-alert email with project warnings

LIST OF TABLES

Table 3.1 simulation-generated risk dataset composition

Table 3.2 Twelve risk prediction features

Table 4.1 Component processing layers

Table 4.2 Operational Contribution Index weights

Table 4.3 AHP criterion priorities

Table 5.1 Main application endpoints

Table 6.1 Classifier test accuracy comparison

Table 6.2 Jira delivery demonstration

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AHP	Analytic Hierarchy Process
API	Application Programming Interface
IPMS	Intelligent Project Management System
Jira	Issue tracking platform
ML	Machine Learning
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol

1 INTRODUCTION

1.1 Overall project context

Team-based software projects are a central part of undergraduate computing education. Students must organise a team, select a feasible topic, develop a plan, implement software, coordinate work and present evidence of progress. A project may appear active at a scheduled review even when work is concentrated among a few students or important tasks are already late. Consequently, supervisors benefit from objective, regularly refreshed information rather than depending entirely on progress presentations.

The Intelligent Project Management System (IPMS) was developed as a group research project with four research components: Intelligent Team Formation and Topic Feasibility; Generative Project Planning and Adaptive Scheduling; Predictive Risk Monitoring and Contribution Analytics; and Automated Quality Assessment and Adaptive Mentorship. The integrated IPMS application provides access to all four components. Each component carries out its own specialised analysis.

1.2 Individual component context

This thesis concerns the third component, Predictive Risk Monitoring and Contribution Analytics. Its purpose is to support the execution stage of a student software project by examining observable work and delivery indicators. The component retrieves GitHub repository activity and, when configured, Jira task data. It calculates a Contribution Index for individual students, prepares a set of project-level risk features and generates a Low, Medium or High risk classification.

The output is presented through project views, dashboards and automated alert emails. A supervisor can inspect inactivity, weekly commits, delayed tasks, contribution distribution and the current risk category. A student or team leader can use the same information to identify where further work or coordination may be needed.

1.3 Problem statement

Manual project supervision can be slow and inconsistent. Commit counts alone do not measure the quality or usefulness of a student's work; similarly, self-reported task completion may omit actual repository activity. When evidence is distributed across source control and issue tracking, supervisors must spend time reconciling information. Without early alerts, a project may remain inactive for an extended period before intervention occurs.

The research problem is therefore to develop a practical mechanism that combines available software activity and delivery information into interpretable individual

contribution indicators and project risk warnings, while remaining usable when some external sources are unavailable.

1.4 Research gap

General-purpose repository dashboards show commits, pull requests and issues, while issue trackers display task progress. These tools do not automatically provide a single academic view that combines individual contribution scoring with project-level risk classification and supervisor-oriented alerts. The contribution of this work lies in bringing these measurements together in one undergraduate project-monitoring workflow.

An additional methodological challenge is that labeled historical datasets of undergraduate project failure or delay are difficult to obtain. This work therefore evaluates a classifier using a simulation-generated dataset with defined risk labels, then demonstrates the operational pipeline with repository and issue-tracker data. The two forms of evaluation answer different questions: simulation-generated testing measures how consistently the model reproduces the constructed classes, whereas operational demonstrations show whether data acquisition and presentation work.

1.5 Aim and objectives

The aim is to design, implement and evaluate an intelligent component that monitors student software project contributions and identifies project risk using GitHub activity, optional Jira delivery metrics and machine learning.

- Develop a multi-criteria Contribution Index for individual project members.
- Extract and prepare project activity and delivery features from GitHub and Jira.
- Train and compare supervised classifiers for three-level project risk classification.
- Implement a dashboard and project management workflow for supervisors and students.
- Generate automated email notifications for projects requiring attention.
- Evaluate model classification results and demonstrate the application using repository and issue-tracker data.

1.6 Research questions or hypotheses

RQ1: How can repository and task activity be combined to provide a more informative contribution score than commit counts alone? RQ2: How effectively do selected supervised learning models distinguish the three defined risk classes in the constructed dataset? RQ3: Can a web application collect available GitHub and Jira metrics and present understandable project warnings to its intended academic users?

The evaluation examines these questions using criterion-weighted scores, classifier metrics, source-code functionality and demonstrated application outputs.

1.7 Scope limitations and assumptions

The component is designed for undergraduate software projects that use GitHub and may use Jira. Jira is optional; in GitHub-only operation, some delivery measures cannot be inferred from issue tracking and the application uses available metrics or supported fallback inputs. A project may have useful work outside GitHub, so activity measures are indicators rather than direct measures of effort or learning.

The simulation-generated classifier labels follow predefined conditions. Consequently, measured test accuracy reflects classification of those constructed labels and should not be interpreted as proven prediction of future academic project failure.

1.8 Individual contribution

My individual research and development focus was the design of contribution analytics and risk prediction, preparation and evaluation of the risk dataset, implementation of the corresponding web application modules and presentation of the results through dashboards and alerts. The wider IPMS project also includes the other three independently operating research components developed by the group.

1.9 Thesis structure

Chapter 2 reviews relevant methods and systems. Chapter 3 presents the research design and evaluation. Chapter 4 explains requirements and solution design. Chapter 5 describes implementation and testing. Chapter 6 reports results, Chapter 7 discusses their meaning, Chapter 8 examines adoption and sustainability, and Chapter 9 concludes the thesis.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review is organised around five relevant areas: collaborative software development in education, mining software repositories, multi-criteria contribution assessment, software project risk prediction and academic dashboards. It is a focused conceptual review rather than a systematic literature review with a formal search protocol.

Published methods and technical documentation provide the background for the selected techniques. The comparison concentrates on their usefulness for an undergraduate project rather than on broad claims about industrial predictive performance.

2.2 Domain and theoretical foundations

Mining software repositories examines development events such as commits, pull requests, reviews and issues to understand how software evolves and how contributors interact. GitHub's REST API provides a programmatic means of retrieving many of these events [1]. Jira's REST API can supply issue status, priority and dates where access has been configured [2]. Combining sources provides a wider view than using commits alone.

Multi-criteria decision analysis supports aggregation of different performance dimensions. The Analytic Hierarchy Process, introduced by Saaty, uses pairwise comparisons to derive relative priorities [3]. In this project, AHP-informed research priorities were studied separately from the operational CI weights implemented in the prototype. Logistic Regression offers an interpretable baseline for multi-class classification; decision trees and ensemble methods provide nonlinear alternatives [4]. Saaty also explains the role of pairwise comparison and consistency in AHP-derived priorities [11].

2.3 Related research

Research on software repository mining demonstrates that development traces can support project analytics, although raw event counts are sensitive to work style, repository conventions and role differences. In educational assessment, peer ratings and supervisor observations can capture communication and conceptual contributions that repositories do not record. The practical design implication is to use multiple signals and to treat automated scores as decision support rather than final grading. Kalliamvakou et al. documented important biases and interpretation challenges in

GitHub data, reinforcing the need to treat repository activity as an indicator rather than a complete measure of student effort [9].

Software defect and effort prediction studies commonly compare statistical and machine-learning approaches using precision, recall, F1-score and confusion matrices. Their evaluation principles are relevant here, but datasets and target labels vary considerably. This thesis therefore reports its own dataset construction and avoids direct accuracy comparisons across unrelated prediction tasks. Breiman introduced Random Forests [12], and Friedman established gradient boosting as an additive predictive modeling approach [13].

2.4 Existing systems products and techniques

GitHub provides repository-level contribution histories and collaboration events; Jira provides issue-tracking and workflow views. Project dashboards can display progress, but their default reports are not specifically designed to assign undergraduate student contribution indices or provide a three-level academic risk classification. Dabbish et al. examined transparency and collaboration on GitHub, supporting the inclusion of collaboration indicators alongside commit activity [10].

IPMS adds a weighted member score, a risk feature vector, a trained classifier and academic warning messages in a single component. The application includes project registration, roster upload and role-based project views to reduce repeated manual setup.

2.5 Gap synthesis and conceptual framework

The identified design gap is not the absence of repository or task-management software. It is the lack of a unified academic monitoring workflow that translates multiple observable signals into both member-level contribution information and project-level intervention cues.

The conceptual framework is: GitHub and optional Jira observations → preprocessing and normalization → contribution criteria and risk features → weighted CI and ML risk category → dashboard and email alerts. The framework explicitly separates member scoring from project classification because these outputs serve different decisions.

3 RESEARCH METHODOLOGY

3.1 Research design

The study uses a quantitative design-and-evaluation approach. The product element is a working monitoring component. The analytical element consists of weighted contribution scoring and supervised classification of project risk. Development and evaluation were iterative: define observable measures, prepare data, compare models, save the selected model and connect inference to a web interface.

Two evaluation settings are distinguished. The classifier comparison uses a labeled simulation-generated dataset. Application demonstrations use GitHub repositories and configured Jira projects or fallback delivery values. These settings provide complementary evidence but are not interchangeable.

3.2 Data participants or study materials

The risk prediction dataset contains 4,000 simulation-generated records representing combinations of repository activity, issue-tracker status and project progress. Its reported class distribution is 1,458 High Risk, 1,403 Medium Risk and 1,139 Low Risk. The dataset is split into 3,200 training and 800 testing observations. No claim of a survey sample or a population-based participant sample is made for this experiment. [7]

Table 3.1 simulation-generated risk dataset composition

Dataset property	**Reported value**
Total records	4,000
Training records	3,200
Testing records	800
High Risk	1,458
Medium Risk	1,403
Low Risk	1,139
Split	80:20

Live application testing used GitHub repository data, including the repository DulakshanaMalith/R26-IT-095, and Jira projects configured for demonstration. The ZIP archive contains the combined dataset, training/test arrays, model files, notebook files and evaluation graphics.

3.3 Experimental or study procedure

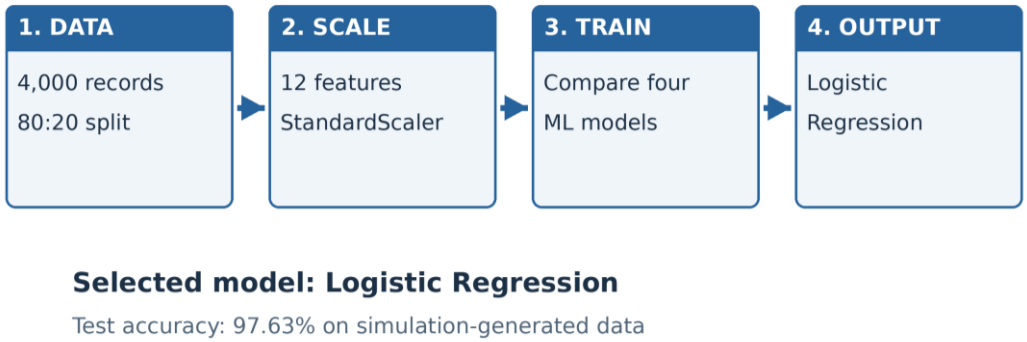


Figure 3.1. Risk classification development and inference workflow.

First, relevant GitHub and Jira indicators were selected from the project requirements. Second, a combined simulation-generated dataset was assembled and checked for class distribution, missing values and duplicates. Third, 12 input features were selected and scaled using StandardScaler. Fourth, Logistic Regression, Decision Tree, Random Forest and Gradient Boosting were trained and evaluated using the saved train/test split. Fifth, the selected classifier and scaler were saved for use in the FastAPI application.

The operational workflow starts when a user registers a project or selects a repository. The application fetches available activity, prepares member and project indicators, calculates CI scores and constructs the 12-feature risk vector. The saved scaler transforms the vector before the classifier produces a risk category. The dashboard presents the category with progress indicators and warnings.

3.4 Measures and evaluation metrics

Classifier accuracy is the proportion of correct labels on the test set. Precision measures the proportion of predicted members of a class that are correct; recall measures the proportion of actual members of a class recovered; F1-score is the harmonic mean of precision and recall. The confusion matrix shows where the model assigns one class instead of another. Weighted F1 accounts for the number of examples in each class.

Table 3.2 Twelve risk prediction features

Feature	Interpretation
total_commits	Repository commits in the observation scope
weekly_commits	Commits in the recent week
inactive_days	Days since relevant activity
overdue_tasks	Tasks past their due date
bug_count	Recorded bug issues
high_priority_bugs	Bug issues marked high priority
task_completion_rate	Percentage of tasks completed
code_quality_score	Derived code-quality indicator
collaboration_score	Derived collaboration indicator
overall_ci_score	Team contribution indicator
progress_percentage	Estimated project progress
delay_count	Late and overdue delivery events

The CI is reported on a 0–100 scale using normalized criterion scores. Risk classifications are High, Medium or Low. Dashboard warning rules provide additional readable triggers, including inactivity, absence of recent commits, overdue work and concentrated participation.

3.5 Data analysis

Exploratory analysis examined feature distributions, missing records, duplicates, class counts and relationships among the selected indicators. The saved analysis report presents histograms, boxplots, model classification reports and confusion matrices. The selected Logistic Regression model achieved 0.97625 accuracy, equivalent to 97.625%, conventionally reported as 97.63%. This corresponds to 781 correct classifications out of 800 test observations.

The comparative results were 97.63% for Logistic Regression, 94.00% for Gradient Boosting, 92.38% for Random Forest and 90.25% for Decision Tree. These results describe the constructed evaluation dataset.

3.6 Ethics privacy security and responsible AI

Repository usernames and project activity can identify students, and issue trackers may contain private project information. The application therefore uses account authentication, project membership records and role-based access concepts. Credentials and API tokens should be kept in environment configuration rather than committed to source control.

Automated contribution scores must be interpreted cautiously. A student may contribute through design, research, testing or discussion without generating a proportional number of commits. A risk alert should prompt review and support, not automatically determine grades or disciplinary outcomes. For this thesis, use of generative AI to assist with drafting is disclosed in Appendix B and remains subject to institutional policy.

3.7 Reproducibility and research artefacts

The source package contains combined_dataset.csv, model_train.ipynb, data_analysis.ipynb, saved train/test arrays, the selected model file and scaler, accuracy comparison results, and classification report/confusion matrix images. The implementation package includes FastAPI backend modules and a React/Vite frontend.

Reproduction requires a compatible Python environment, the dependencies listed in requirements.txt, project configuration, and API credentials for live GitHub/Jira calls. simulation-generated classification can be reproduced from the saved datasets and model notebooks independently of the availability of live API credentials.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The primary functional requirements are project registration and access control, GitHub repository association, optional Jira association, bulk team roster upload, member contribution calculation, project risk classification, dashboard display and risk-alert email generation. The user interface should make risk and activity indicators understandable to a supervisor without requiring direct inspection of API responses.

Non-functional requirements include reasonable response times, predictable fallback behavior when Jira is not configured, secure handling of credentials, understandable error messages and maintainable separation of data acquisition, scoring, prediction and presentation modules. The prototype includes a 120-second in-memory dashboard cache to reduce repeated external requests.

4.2 Component architecture within the complete solution

IPMS provides one application entry point to four independently operating research components. The Risk component is responsible for monitoring and contribution analysis; it does not require a direct data feed from the other three components to calculate its scores. Project identifiers, repository URLs and optional Jira keys provide the local context for analysis.

The Risk component's processing pipeline is D → P → F → {CI, R} → V, where D denotes data acquisition, P preprocessing, F feature extraction, CI Contribution Index calculation, R risk classification and V visual presentation. A notification service uses the same project indicators to prepare alerts.

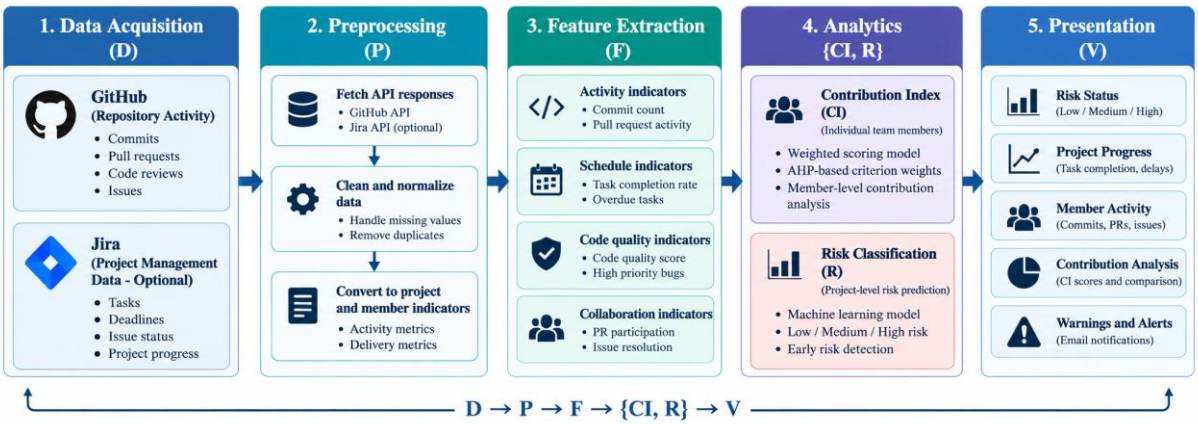


Figure 4.1. Architecture of the Predictive Risk Monitoring and Contribution Analytics component.

Table 4.1 Component processing layers

Layer	Main responsibility
Input	Project records, GitHub REST API and optional Jira REST API
Preprocessing	Clean and normalize activity and delivery indicators
Analytics	Weighted CI and saved ML risk classifier
Presentation	FastAPI templates, React frontend and JSON endpoints
Notification	SMTP risk-alert emails
Persistence	SQLite users, projects, memberships, sessions and alert records

4.3 Design alternatives and decisions

A simple commit-count score was rejected as the sole contribution indicator because it overlooks task delivery, collaboration and code quality. A weighted multi-criteria CI was selected to represent multiple dimensions. Two weight sets appear in the research artefacts: operational prototype weights and a separate AHP-based analysis. They are reported distinctly to avoid presenting a research prioritization exercise as though it were the exact deployed scoring configuration.

For project risk, four standard classifiers were compared rather than selecting a complex model by default. Logistic Regression was retained because it performed best on the constructed test dataset and is straightforward to deploy with scikit-learn.

4.4 Detailed design

The implemented Contribution Index uses five normalized criteria: Commit Activity 30%, Task Completion 25%, Collaboration 20%, Issue Resolution 15% and Code Quality 10%. For a member i, $CI_i = 0.30\ C_i + 0.25\ T_i + 0.20\ L_i + 0.15\ I_i + 0.10\ Q_i$, where each criterion is on a common 0–100 scale. When a source criterion is unavailable, the implementation supports dynamic renormalization of available weights. [5], [8]

Table 4.2 Operational Contribution Index weights

Operational CI criterion	Weight
Commit Activity	30%
Task Completion	25%

Collaboration	20%
Issue Resolution	15%
Code Quality	10%

The separate AHP-based research priority set is Commit Activity 22.7%, Task Completion 21.8%, Code Quality 19.9%, Activity Consistency 19.0% and Collaboration 16.6%. The different criteria and priorities indicate that this is an analytical comparison, not the same formula used by the operational CI implementation.

Table 4.3 AHP criterion priorities

AHP criterion	Research priority
Commit Activity	22.7%
Task Completion	21.8%
Code Quality	19.9%
Activity Consistency	19.0%
Collaboration	16.6%

Risk inference uses 12 features in a fixed order. The feature vector is transformed using the saved scaler and passed to the Logistic Regression model. For Jira delivery reporting, completion is Done / Total × 100; progress is (Done + 0.5 × In Progress) / Total × 100. Delays include completed-late tasks and overdue active tasks.

Dashboard warnings include a High or Medium risk category, inactivity above 14 days, no weekly commits, high-priority bugs, overdue tasks, delays and concentration of more than 60% of recorded contribution in one member.

4.5 Standards and professional practice

FastAPI provides documented HTTP route handling and JSON APIs, while GitHub and Jira are accessed through their published REST interfaces. Scikit-learn and joblib are used for model training and persistence. Application credentials are configured outside the tracked codebase. Academic reporting separates training data, test metrics, operational demonstrations and user-facing warnings so that each form of evidence is interpreted appropriately.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The backend uses Python, FastAPI, Uvicorn, requests, scikit-learn and joblib. HTML templates and static styles support server-rendered views; a React/Vite frontend provides additional application pages and uses /api routes. SQLite stores users, projects, memberships, sessions and risk-alert records. Excel bulk-upload support is implemented for team registration. [8]

The main implementation files include main.py, contribution.py, jira_client.py, database.py, auth.py, bulk_upload.py, risk_monitor.py and emailer.py. The ZIP also includes trained model artefacts, notebook files and accuracy-report images.

5.2 Component implementation

Project management endpoints support listing projects, creating a project, editing project details, adding or removing members, and uploading roster information from Excel. Authentication routes support registration, login, logout and current-user lookup. The frontend's API client sends credentialed requests to /api endpoints.

Contribution calculation retrieves repository events and constructs normalized member indicators. Dashboard generation retrieves project metrics, evaluates risk and warnings, and presents member scores, recent activity, delivery tiles and alert information. The risk model is loaded from a saved joblib file and applied to the scaled 12-feature input.

Table 5.1 Main application endpoints

HTTP method	Representative route
GET	/projects
POST	/projects/upload
GET / POST	/contribution
GET	/dashboard
GET	/api/dashboard-data
GET	/api/jira-metrics
POST	/predict
GET	/api/projects
POST	/api/projects
POST	/api/test-email

Automated alert logic assembles the risk level, project indicators, active warnings and a dashboard link into an email. A supplied Gmail screenshot shows a delivered message titled '[IPMS] ExceptionGroup Port is HIGH RISK', addressed to student and supervisor recipients, containing inactivity and contribution indicators.

5.3 Integration with the other components

The final IPMS application makes the four research components accessible through one interface. Team Formation and Topic Feasibility, Project Planning and Scheduling, Risk Monitoring and Contribution Analytics, and Quality Assessment and Adaptive Mentorship remain independently operating analytical components. Their common application access supports a unified user experience without requiring direct cross-component data exchange.

The Risk component integrates internally with its own project records, authentication, GitHub/Jira data retrieval, classifier and notification logic. A running application was accessed at http://127.0.0.1:5000; local URLs are included here as development environment examples rather than public deployment addresses.

5.4 Verification and software testing

Model testing used the held-out simulation-generated test partition and produced classifier accuracy, precision, recall, F1 and confusion matrices. Functional demonstrations exercised the contribution form, dashboard, prediction screen and automated alert delivery. Screenshots show repository-driven member contribution and project warning outputs.

A seeded Jira demonstration contained 12 tasks: five Done (three completed on time and two late), three In Progress (two overdue) and four To Do (one overdue). This yielded a 41.67% completion rate, 54.17% progress, three overdue tasks and five delays. These figures provide a transparent example of the delivery calculations.

The dashboard demonstration for the repository R26-IT-095 showed High Risk, 96 inactive days, a team average CI of 63.5, and no Jira progress data in GitHub-only operation. The separate prediction form example showed High Risk with 98 inactive days and manually supplied delivery values after no issues were returned for a selected Jira project. These are different demonstration states, not contradictory measurements of one fixed run.

5.5 Deployment and operation

The application is run locally with the Python dependencies installed and Uvicorn serving the FastAPI application. The React frontend can be run separately during

development and connects to the backend API. The running integrated application shown in the user demonstration used localhost port 5000.

To operate a project, the user signs in, creates or selects a project, supplies a GitHub repository URL, optionally supplies a Jira project key, and opens the contribution or dashboard view. A team template supports batch registration. SMTP configuration enables the alert email workflow. GitHub API rate limits and missing Jira configuration are handled through the available source-specific behavior.

6 RESULTS AND VALIDATION

6.1 Research results

On the simulation-generated 800-record test set, Logistic Regression achieved the highest reported accuracy at 97.63%. The weighted F1-score was approximately 0.98. Gradient Boosting, Random Forest and Decision Tree followed at 94.00%, 92.38% and 90.25%, respectively. [7]

Table 6.1 Classifier test accuracy comparison

Model	**Test accuracy**
Logistic Regression	97.63%
Gradient Boosting	94.00%
Random Forest	92.38%
Decision Tree	90.25%

The strong performance is consistent with the use of threshold-derived simulation-generated labels. It demonstrates that the trained model can learn the constructed classification patterns. It should not be presented as a prospective accuracy measurement on independently labeled real undergraduate project outcomes.

6.2 Product and technical results

The working component presents project information, member Contribution Index values, recent commits, inactivity, risk status and warning messages. Its project-management functionality includes project records, authentication and Excel roster upload. The source code also implements automated email alerts.

Table 6.2 Jira delivery demonstration

Jira demonstration metric	**Value**
Total tasks	12
Done	5
In Progress	3
To Do	4
Completion rate	41.67%
Progress estimate	54.17%
Overdue tasks	3

Delay events	5

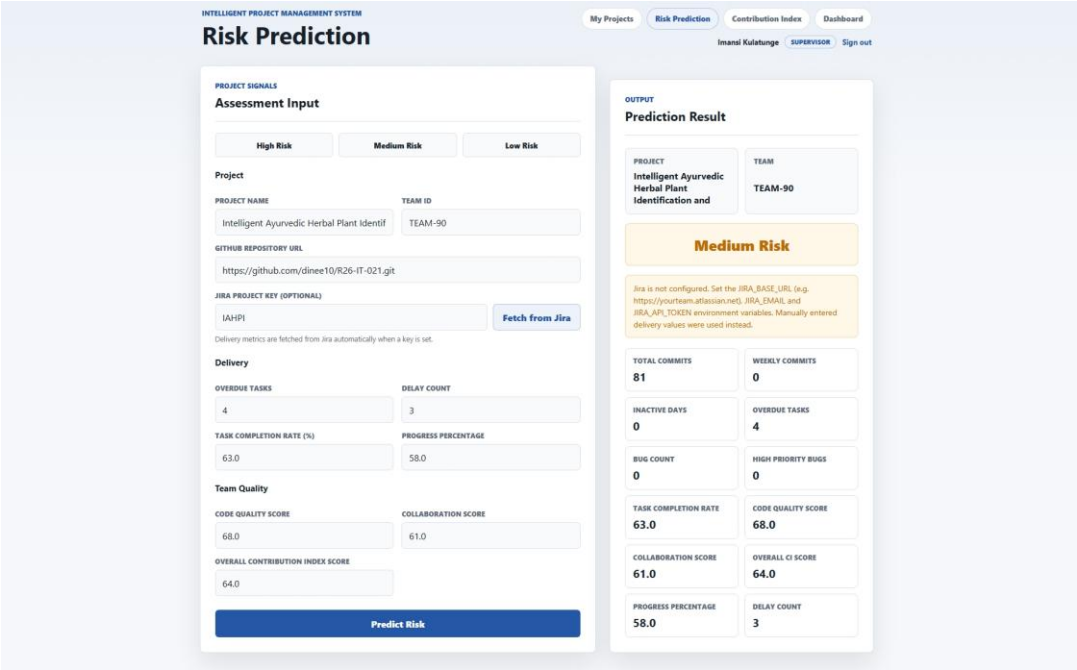


Figure 6.1. Risk prediction interface with Medium Risk classification.

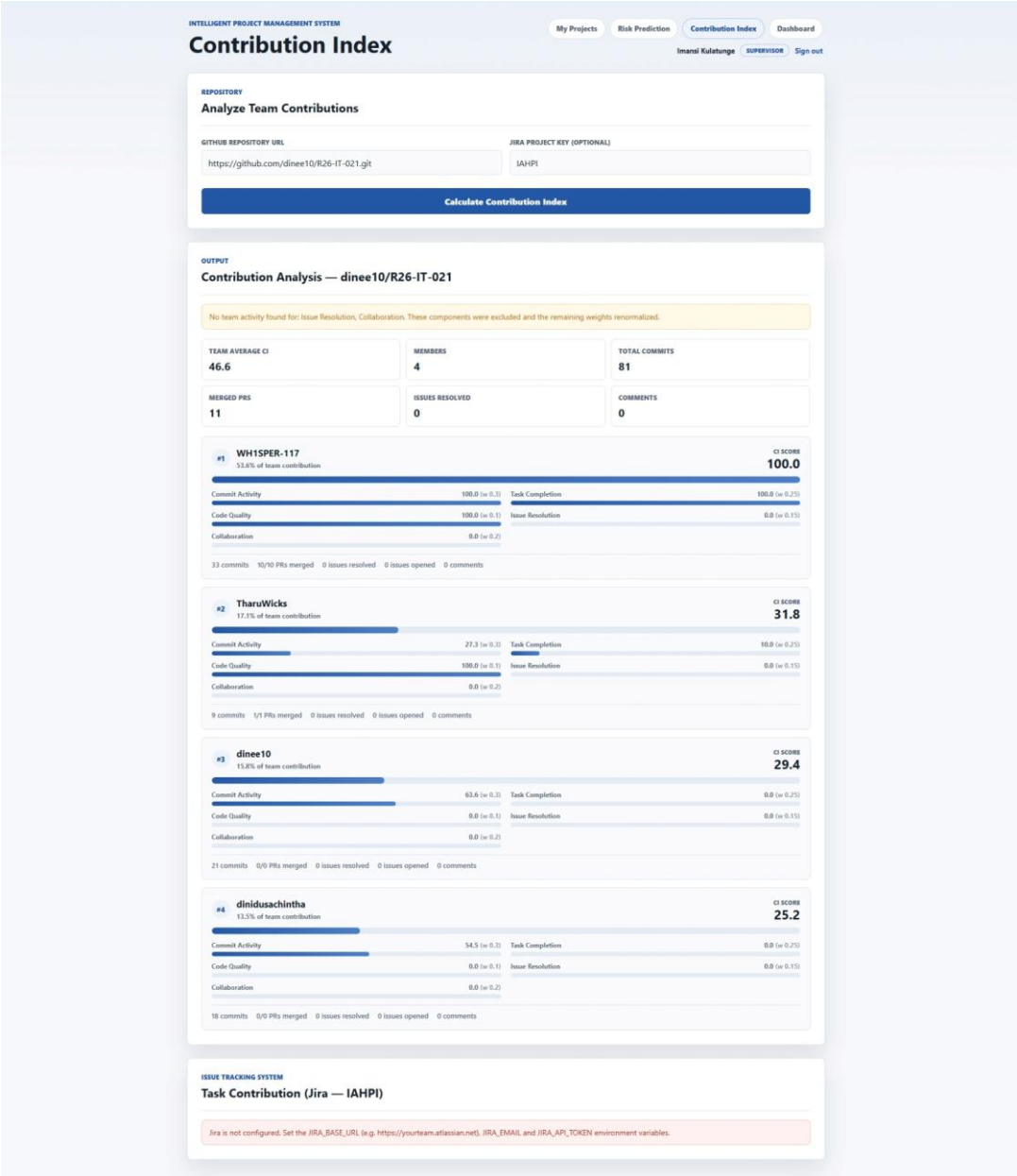


Figure 6.2. Contribution Index results for a four-member project.

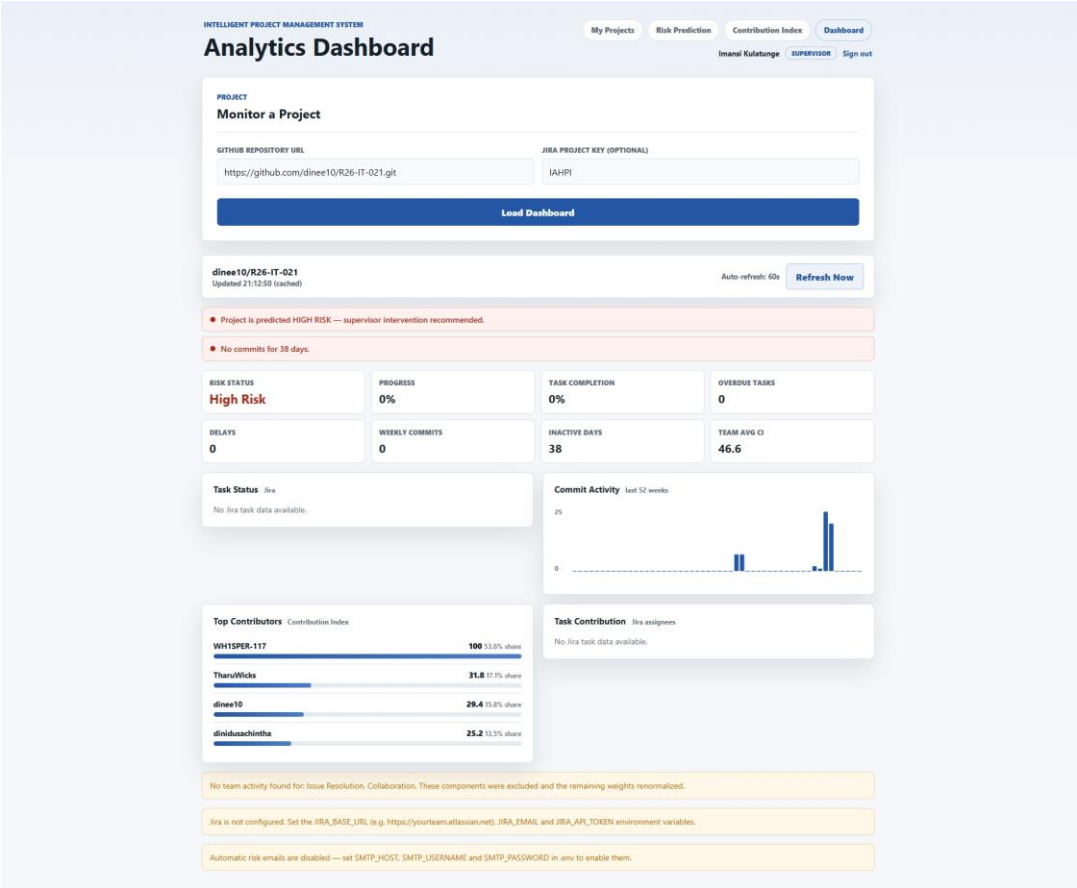


Figure 6.3. Analytics dashboard showing High Risk and project activity warnings.

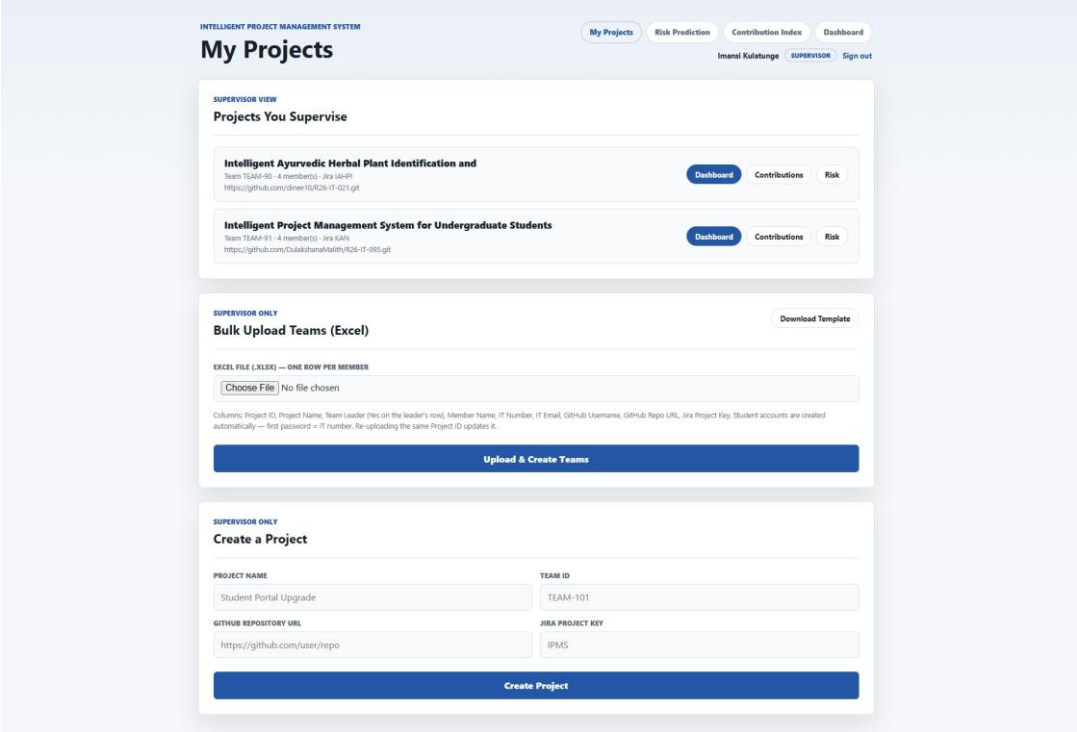


Figure 6.4. Project listing and Excel bulk upload interface.

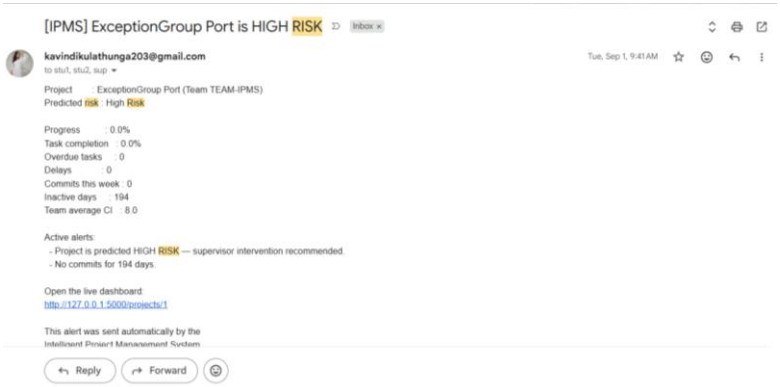


Figure 6.5. Delivered automated risk-alert email with project risk and inactivity warnings.

6.3 User or stakeholder validation

The functional interface was demonstrated through screenshots of project analytics, contribution calculation, risk prediction and a delivered email notification. The alert email included the project name, predicted risk, inactivity days, average CI and direct dashboard link. These outputs demonstrate that the selected indicators can be communicated to intended users in a concise operational format.

An expert evaluation form was prepared as part of the research workflow, but no aggregate expert rating or formal user-study outcome is included in the supplied quantitative results. Therefore, this thesis does not invent a usability score or claim a statistically evaluated user-acceptance result.

6.4 Integration results

The component operates within the common IPMS application alongside the other three research components. Its internal workflow connects project records, GitHub/Jira retrieval, CI calculation, risk classification, dashboards and email alerts. The delivered alert message links to the project's page on the running local application.

The four research components are accessible from the same integrated application but do not exchange analytical data directly. This architecture provides shared access while preserving each component's independent processing logic.

6.5 Summary of findings

The research produced a five-criterion contribution scoring model, a 12-feature project risk classifier and a functioning monitoring interface. Logistic Regression gave the strongest result among the four compared models on the constructed dataset. Repository and Jira demonstrations illustrated operational indicators, and an email screenshot documented alert delivery.

The main finding is that a practical academic monitoring workflow can combine software activity, contribution scoring, classification and actionable warnings. The predictive value of its labels for future real-world project outcomes remains a separate research question.

7 DISCUSSION

7.1 Interpretation of findings

The Contribution Index provides a structured alternative to counting commits alone. Its weights make the scoring policy explicit and allow member activity to be compared on normalized criteria. This transparency is useful for discussing uneven participation, but the score remains dependent on the events that the underlying tools record.

The classification result demonstrates that Logistic Regression was sufficient for separating the constructed risk categories. A more complex ensemble model did not outperform it in the reported test. Since labels were generated using defined conditions, the result is best interpreted as successful approximation of those conditions rather than proof of general project-outcome forecasting.

7.2 Comparison with literature baselines and existing solutions

GitHub and Jira provide detailed operational records, but supervisors must interpret their separate metrics. The component adds academic-specific aggregation and warning logic. In contrast to a simple activity chart, it presents a member contribution score and a project risk category together.

The research does not claim superiority over a published external model evaluated on a different dataset. The four-model comparison uses one common local evaluation dataset, making the within-study comparison meaningful while avoiding unsupported cross-study accuracy claims.

7.3 Research contribution

The main research contribution is an operational framework that combines weighted contribution analytics and supervised project risk classification in the context of undergraduate software teams. The framework defines measurable indicators, documents an interpretable CI formula, compares classifiers and connects the selected model to a working application.

The AHP criterion-priority study also contributes a structured way of examining which activity dimensions experts may value. Its research priorities differ from the deployed prototype weights, providing a clear opportunity for further empirical calibration.

7.4 Product contribution

The implemented product includes project registration, roster upload, role-based accounts, GitHub and optional Jira retrieval, dashboard analytics, saved-model inference and automated email alerts. The combination makes monitoring results accessible without requiring users to run model notebooks or inspect raw API data.

The component's modular backend and frontend files also support maintainability. Independent operation within the common IPMS application allows the monitoring workflow to be demonstrated using its own repository and issue-tracker configuration.

7.5 Limitations and threats to validity

The principal internal-validity limitation is that the simulation-generated risk labels follow predefined conditions. A classifier may learn those conditions very accurately without predicting an independently observed future outcome. The study also uses a single 80:20 split; additional repeated validation would strengthen confidence in the stability of the reported comparison.

Construct validity is affected by the use of repository and issue-tracker proxies. Commit quantity does not fully measure quality, creativity or effort. Jira completeness depends on the team's issue-tracking discipline. The selected CI weights express a scoring policy and may not be equally appropriate across project types or student roles.

External validity is limited by the absence of a multi-cohort longitudinal evaluation with independently recorded project outcomes. The operational screenshots demonstrate functionality at particular times, not the long-term reliability of the service or the causal effect of alerts on student performance.

7.6 Implications

For supervisors, the system offers a structured way to identify projects requiring closer attention. For students, it can make visible activity patterns and encourage discussion of work distribution. For institutions, the framework illustrates how educational analytics can be added to existing software development tools.

Responsible adoption requires explaining what the metrics represent, allowing contextual review and avoiding automatic grading from CI or risk alone. The results also motivate future collection of real, consented project outcome data.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The principal target users are computing faculties, project supervisors, undergraduate development teams and academic project coordinators. The value proposition is a single monitoring view that reduces manual consolidation of repository and issue-tracker activity and makes potential delays visible through alerts.

An institution could deploy the monitoring component as part of its research-project management workflow. The intended benefit is earlier, more informed intervention, rather than replacement of academic judgement.

8.2 Market and competitive position

GitHub and Jira already provide mature repository and task-management capabilities. IPMS is positioned as an academic analytics layer built around those tools, rather than as a replacement for them. Its distinguishing features are student-level contribution scoring, academic risk classification, roster management and supervisor-oriented notifications.

Institutions evaluating the product would need to consider data access, academic assessment rules, available technical support and the consistency of student tool usage.

8.3 Business or sustainability model

A sustainable deployment could be institution-supported software operated by a faculty or university IT team. The main recurring costs would be hosting, application maintenance, secure storage, API administration, user support and model review. A hosted subscription is another possible distribution model, but no revenue or market-size estimates were measured in this research.

Long-term value depends on keeping the system usable, maintaining GitHub/Jira integrations and training academic staff to interpret scores responsibly.

8.4 Scalability intellectual property and compliance

The prototype uses a local SQLite database and in-memory caching. Larger deployments would require appropriate backup, concurrency, access-control and monitoring arrangements. API usage must respect provider permissions and rate limits. Data retention and student privacy controls should follow institutional policies.

Any external distribution should review the licenses of included libraries and confirm ownership of the group's original code, model artefacts and documentation. Student activity data should be collected and retained only for justified educational purposes.

8.5 Contribution to the Sustainable Development Goals

The work aligns primarily with SDG 4, Quality Education, by supporting the supervision and assessment process of collaborative learning projects. It also relates to SDG 9, Industry, Innovation and Infrastructure, through the design of an applied software analytics tool.

These are qualitative alignments to the goals; the research did not measure a direct numerical SDG impact.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the design, implementation and evaluation of the Predictive Risk Monitoring and Contribution Analytics component of IPMS. The component collects GitHub and optional Jira indicators, calculates weighted member contribution scores, predicts a three-level project risk category and communicates results through dashboards and email alerts.

Among the evaluated classifiers, Logistic Regression achieved the best reported performance on the 4,000-record simulation-generated dataset, with 97.63% test accuracy. Operational demonstrations showed the component's contribution, dashboard and alert workflows. The completed work provides a practical foundation for data-informed supervision of undergraduate software projects.

9.2 Achievement of objectives

The work defined and implemented a five-criterion Contribution Index, selected 12 project risk features, prepared and evaluated four classification models, integrated the selected model into a FastAPI application, and implemented project registration, bulk upload, dashboards and automated email alerts.

The results support the development and technical objectives. Establishing the model's predictive validity against independent real project outcomes remains an appropriate next stage.

9.3 Individual research and product contributions

My individual contribution centred on the risk and contribution research problem: selecting measurable indicators, developing and documenting scoring criteria, preparing risk model experiments, interpreting classification results, and implementing and demonstrating the component's analytics and alert functions.

The larger group created the overall IPMS experience containing four separately operating research components. The risk component contributes the monitoring and early-warning capability within that experience.

9.4 Future directions

Future work should collect longitudinal data from multiple consenting student project teams and define outcome labels independently of the model's input thresholds. Repeated cross-validation and external testing would then allow a stronger assessment of predictive performance. Contribution weights could be calibrated through documented expert and student evaluations.

Further work could investigate repository-independent activities, explainable warning messages, model drift, fairness across student roles, and institution-scale deployment with stronger monitoring and data governance.

REFERENCES

[1] GitHub, “REST API documentation,” GitHub Docs. [Online]. Available: https://docs.github.com/en/rest. Accessed: Oct. 10, 2026.

[2] Atlassian, “The Jira Cloud platform REST API,” Atlassian Developer. [Online]. Available: https://developer.atlassian.com/cloud/jira/platform/rest/v3/. Accessed: Oct. 10, 2026.

[3] T. L. Saaty, The Analytic Hierarchy Process. New York, NY, USA: McGraw-Hill, 1980.

[4] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[5] IPMS Research Group, “IPMS Metrics Reference,” internal project technical document, 2026.

[6] IPMS Research Group, “IPMS Function Guide,” internal project technical document, 2026.

[7] IPMS Risk Component, “Risk Prediction Data Analysis Report,” internal model evaluation report, 2026.

[8] IPMS Risk Component, “component_risk source-code archive,” project implementation, model and test artefacts, 2026.

[9] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “The promises and perils of mining GitHub,” in Proc. 11th Working Conf. Mining Software Repositories (MSR), 2014, pp. 92–101, doi: 10.1145/2597073.2597074.

[10] L. Dabbish, C. Stuart, J. Tsay, and J. Herbsleb, “Social coding in GitHub: Transparency and collaboration in an open software repository,” in Proc. ACM Conf. Computer Supported Cooperative Work (CSCW), 2012, pp. 1277–1286, doi: 10.1145/2145204.2145396.

[11] R. W. Saaty, “The analytic hierarchy process—What it is and how it is used,” Mathematical Modelling, vol. 9, no. 3–5, pp. 161–176, 1987, doi: 10.1016/0270-0255(87)90473-8.

[12] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[13] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

GLOSSARY

Term	Definition
Activity Consistency	The regularity of a student's project-related activities over a specified period.
Analytic Hierarchy Process (AHP)	A structured decision-making technique used to determine the relative importance of contribution evaluation criteria through pairwise comparisons.
Application Programming Interface (API)	A mechanism that enables software applications to exchange data and communicate with external services.
Collaboration Score	A measure of a team member's collaborative activities, such as pull-request participation and other recorded interactions.
Commit Activity	Code changes recorded in a Git repository, used as an indicator of development participation.
Contribution Index (CI)	A weighted score used to evaluate the relative contribution of individual students based on measurable project activities.
Data Preprocessing	The process of cleaning, transforming, and preparing collected data for analysis and machine-learning predictions.
Feature Extraction	The process of converting raw GitHub and Jira data into measurable indicators for contribution analysis and risk prediction.
GitHub	A Git-based software development platform used to retrieve repository activity, commits, pull requests, and issues.
Inactive Days	The number of days since the most recent recorded development activity.
Issue Resolution	The process of addressing and closing reported software issues or defects.
Jira	A project and issue management platform used to track tasks, deadlines, completion status, and project progress.
Logistic Regression	A supervised machine-learning algorithm used in this research to classify undergraduate project risk into Low, Medium, and High categories.
Machine Learning (ML)	A branch of artificial intelligence that enables systems to identify patterns in data and generate predictions.
Overdue Task	A project task that has passed its assigned deadline without being completed.
Pull Request (PR)	A proposed set of code changes submitted for review before being merged into a repository.

Risk Classification	The process of categorizing project conditions into Low, Medium, or High risk using measurable indicators.
Risk Monitoring	The continuous or periodic evaluation of project activities to identify potential delays, inactivity, and other warning conditions.
StandardScaler	A preprocessing technique that standardizes numerical features by centering them around their mean and scaling them using their standard deviation.
Task Completion Rate	The percentage of assigned project tasks that have been completed.
Weighted Scoring Model	An evaluation method that combines multiple criteria according to their assigned importance to calculate an overall score.

APPENDICES

Appendix A Research instruments or extended results

A1. Operational CI weights and research AHP priorities are presented in Tables 4.2 and 4.3. A2. The 12 risk prediction features are presented in Table 3.2. A3. The complete classifier comparison is presented in Table 6.1. A4. Supporting artefacts include the dataset, saved train/test arrays, notebook files, trained model, scaler, model classification reports and confusion matrices. A5. Screenshots of the operational dashboard, prediction interface and delivered alert are reproduced in Chapter 6.

A6. Demonstration risk alert: ExceptionGroup Port (TEAM-IPMS); High Risk; 194 inactive days; team average CI 8.0; 0% progress; 0% task completion; email includes supervisor intervention recommendation and dashboard link.

Appendix B Academic integrity and AI use disclosure

Generative AI assistance: ChatGPT was used to help organise, draft and edit explanatory prose in this thesis, using the author's supplied template, project documents, code, research results and screenshots. The author must review the wording, verify citations and numerical statements against the original research artefacts, and follow SLIIT's disclosure requirements before signing the declaration.

No AI-generated experimental results are claimed. The numerical model results in this thesis are taken from the supplied IPMS Risk Prediction Data Analysis Report. The author remains responsible for checking and approving the final submitted version.